Estrutura de Dados II

Prof. Vanessa Matias Leite

1

Árvores Rubro-Negras

2

Árvores Rubro-Negras

- Uma árvore vermelho-preto é uma árvore de busca binária com um bit extra de armazenamento por nó: sua **cor** ela pode ser VERMELHA ou PRETA.
- Restringindo as cores dos nós em qualquer caminho simples da raiz até uma folha, as árvores vermelho-preto asseguram que o comprimento de nenhum desses caminhos seja maior que duas vezes o de qualquer outro, de modo que a árvore é aproximadamente balanceada.

3

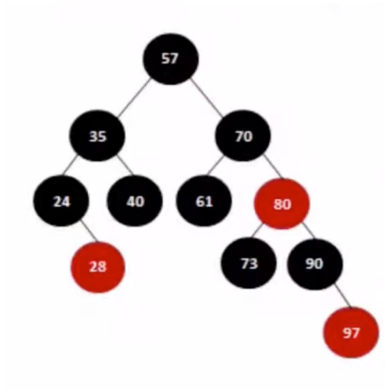
Árvores Rubro-Negras

1. Todo nó é vermelho ou preto.
2. A raiz é preta.
3. Toda folha (NULL) é preta.
4. Se um nó é vermelho, então os seus filhos são pretos.
5. Para cada nó, todos os caminhos simples do nó até folhas descendentes contêm o mesmo número de nós pretos.

4

Árvores Rubro-Negras

1. Todo nó é vermelho ou preto.
2. A raiz é preta.
3. Toda folha (NULL) é preta.
4. Se um nó é vermelho, então os seus filhos são pretos.
5. **Para cada nó,** todos os caminhos simples do nó até folhas descendentes contêm o mesmo número de nós pretos.



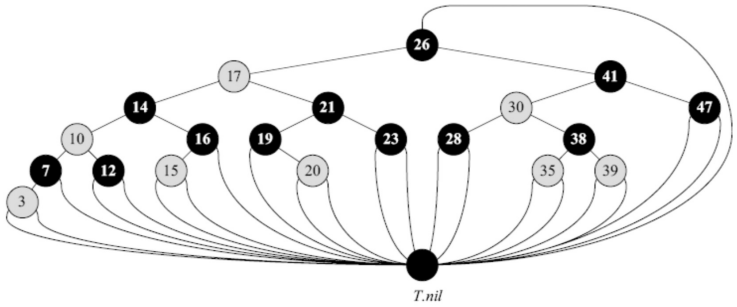
5

Árvores Rubro-Negras

- Folhas nulas são obrigatórias (Regra 3: Toda folha (NULL) é preta).
- Pode-se utilizar um único nó de sentinela para representar todas as folhas (NULL);
- **A folha para as árvores RB são os ponteiros nulos!!!**

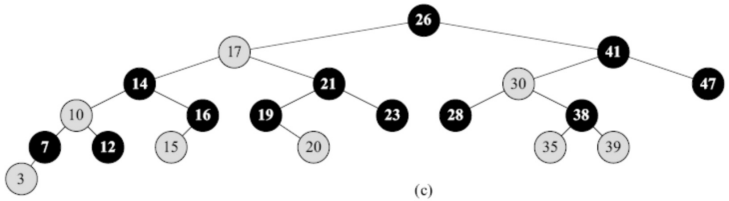
6

Árvores Rubro-Negras



7

Árvores Rubro-Negras



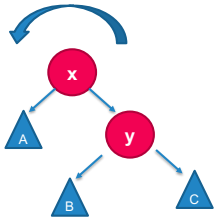
8

Rotações

- Árvores RB também executam rotações para efetuar ajustes;
- Procedimento é semelhantes ao das árvores AVL;
- Rotação simples à esquerda passa a se chamar **rotação para a direita;**
- Rotação simples à direita passa a se chamar **rotação para a esquerda;**

9

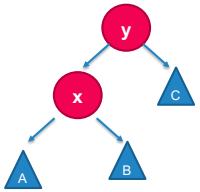
Rotação RB para a esquerda



- Trata-se de uma rotação entre os nós x e seu filho à direita, y, envolvendo as subárvores A, B e C.

10

Rotação RB para a esquerda



- Trata-se de uma rotação entre os nós x e seu filho à esquerda, y, envolvendo as subárvores A, B e C.

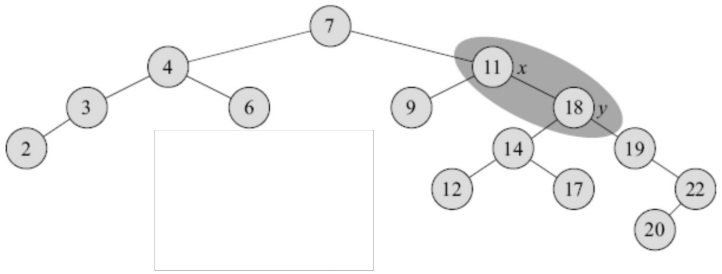
11

Rotação RB para a esquerda

```
LEFT-ROTATE(T; x)
    y = x.direita        // define y
    x.direita = y.esquerda // transforma a subárvore à esquerda de y na subárvore à direita de x
    if y.esquerda ≠ T.nil
        y.esquerda.p = x
    y.p = x.p        // liga o pai de x a y
    if x.p == T.nil
        T.raiz = y
    elseif x == x.p.esquerda
        x.p. esquerda = y
    else x.p.direita = y
    y.esquerda = x     // coloca x à esquerda de y
    x.p = y
```

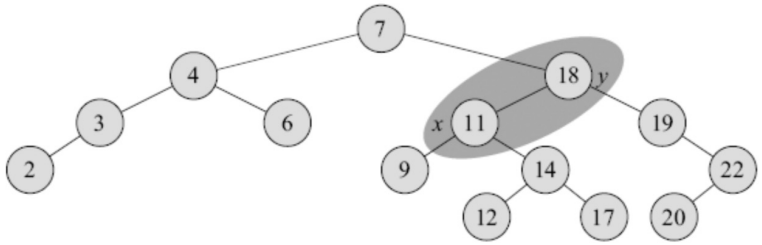
12

Rotação RB para a esquerda



13

Rotação RB para a esquerda



14

Inserções na Árvore RB

- A inclusão é executada utilizando o algoritmo de inserção em árvore de busca binária;
- Modificado para manter os ponteiros do pai;
- Todo novo nó ser VERMELHO;
- Sempre ganha duas folhas nulas PRETAS;

15

Inserções na Árvore RB

- Se o pai do novo nó for PRETO (Caso 0):
  - ✓ Está pronto, trata-se de uma árvore RB;
- Se o pai for VERMELHO:
  - ❑ A propriedade 4 está ferida (Se um nó é vermelho, então os seus filhos são pretos).
  - ❑ Até duas rotações devem ser realizadas

16

Ajustes nas Árvores RB

- Vamos definir uma vizinhança:
  - v (vértice), seu pai, seu avô e seu tio (RB é binária, portanto só haverá no máximo um tio);
- Passo-CE (Correção ou Elevação):
  - Recolorir
  - Rotacionar a vizinhança (eventualmente);

17

Ajustes nas Árvores RB

- Com um Passo-CE podemos ter:
  - Restituição da Regra 4 (Se um nó é vermelho, então os seus filhos são pretos);
  - Elevação de um erro para um nível acima na árvore;
- Para todo Passo-CE:
  - Existem 6 casos possíveis;
  - 3 para a direita e 3 para a esquerda;

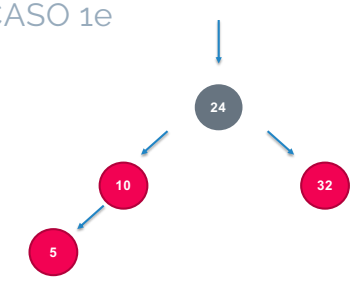
18

CASO 1e

- v possui um tio vermelho e seu pai é um filho rubro da esquerda;
  - Não importa se v é filho da esquerda ou da direita;
- Recolorir o pai, avô e tio de v;
  - Se o bisavô de v for preto, a árvore fica correta.
- Caso contrário, a regra 4 foi violada e elevada a dois níves:
  - v é elevado dois níveis;
  - executa-se um Passo-CE para o novo v (antigo avô de v)

19

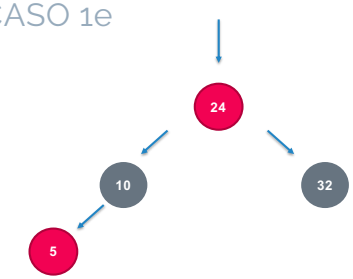
CASO 1e



- E se eu inserir o valor 5 ?

20

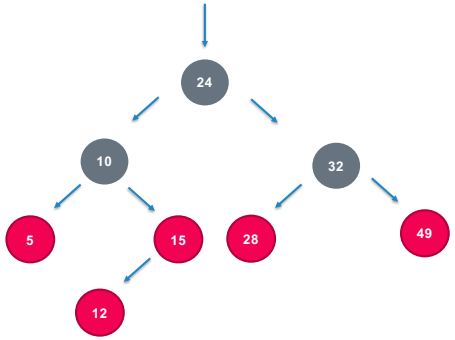
CASO 1e



- Recolorir os dois irmãos vermelhos e o pai preto
- Se 24 for a raíz deveremos aplicar outra regra (veremos mais a seguir)
- Se 24 não for raiz, subir a verificação

21

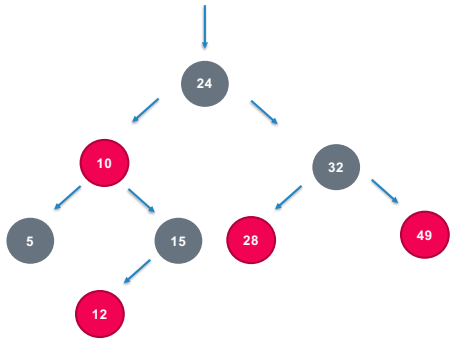
CASO 1e



- E se eu inserir o 12?

22

CASO 1e



- E se eu inserir o 12?
- Troco a cor do pai, do tio e do avó.
- Casos mais simples, pois só mudo a cor do nó!

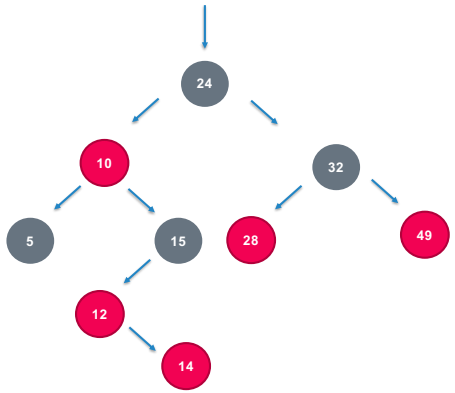
23

Caso 2e

- v é um filho da direita, cujo o pai é um filho rubro da esquerda e cujo o tio é negro;
- Rotacionamos para esquerda v e seu pai
  - Atualizamos o nó corrente;
  - Caímos no Caso 3;
- O Caso 2 (inserção) sempre é sucedido pelo Caso 3;
- O Caso 3 pode acontecer sozinho.

24

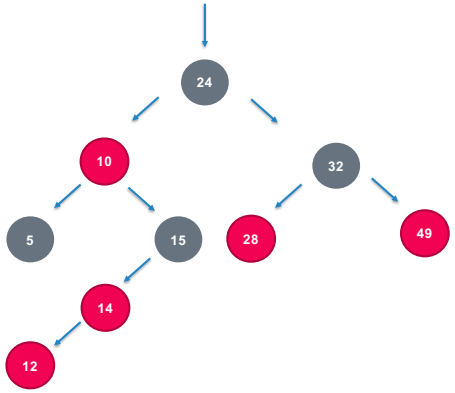
Caso 2e



- E se eu adicionar o 14?
- Tio de 14 é um nó preto!
- Folha nula sempre é preta!

25

Caso 2e



- Rotacionamos para a esquerda!
- Como consequência, geramos o Caso 3

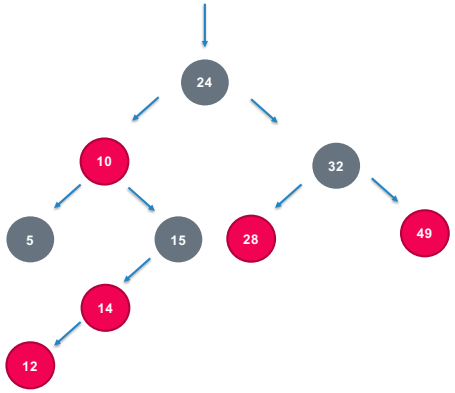
26

Caso 3e

- v é um filho da esquerda, cujo o pai é um filho vermelho da esquerda e cujo o tio é preto.
- Fazer a rotação a direita entre o avô e o pai;
- Recolorimos o pai e o avô:
  - ✓ O avô fica vermelho;
  - ✓ O pai fica preto;

27

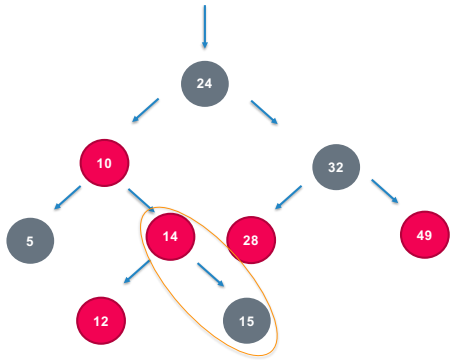
Caso 3e



- Rotacionar a direita;
-

28

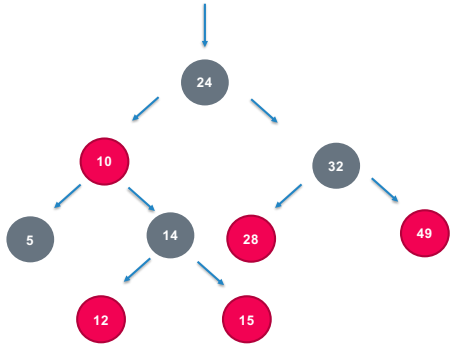
Caso 3e



- Rotacionar a direita;
- Recolorir pai e o avô:
  - ✓ O avô fica vermelho;
  - ✓ O pai fica preto;
-

29

Caso 3e



- Rotacionar a direita;
- Recolorir pai e o avô:
  - ✓ O avô fica vermelho;
  - ✓ O pai fica preto;
- Lembrando: O Caso 3 pode acontecer sozinho. O Caso 2 necessita do Caso 3!

30

```
RB-INSERT(T,z)
y = T.nil
x = T.raiz
while x ≠ T.nil
    y = x
    if z.chave < x.chave
        x = x.esquerda
    else x = x.direita
z.p = y
if y == T.nil
    T.raiz = z
elseif z.chave < x.chave
    y.esquerda = z
else y.direita = z
z.esquerda = T.nil
z.direita = T.nil
z.cor = RED
RB-INSERT-FIXUP(T,z)
```

```
RB-INSERT-FIXUP(T, z)
while z.p.cor == VERMELHO
  if z.p == z.p.p.esquerda
      y = z.p.p.direita
      if y.cor = VERMELHO
          z.p.cor = PRETO                    // caso 1
          y.cor = PRETO                      // caso 1
          z.p.p.cor = VERMELHO               // caso 1
          z = z.p.p                          // caso 1
      else if z = z.p.direita
              z = z.p                        // caso 2
              LEFT-ROTATE(T, z)              // caso 2
          z.p.cor = PRETO                    // caso 3
          z.p.p.cor = VERMELHO               // caso 3
          RIGHT-ROTATE(T, z.p.p)             // caso 3
      else (igual à cláusula then
          com "direita" e "esquerda" trocadas)
T.raiz.cor = PRETO
```

31

Exclusão em Árvores Rubro-Negras

- Mais complexa que inserção;
- Ao inserir um nó, uma violação vermelha é forçada e, então, corrigida. Tais violações vermelhas são relativamente fáceis de corrigir.
- Quando se remove um nó, não se tem a opção de escolher sua cor.

32

Exclusão em Árvores Rubro-Negras

- REGRA 5. Para cada nó, todos os caminhos simples do nó até folhas descendentes contêm o mesmo número de nós pretos.
- A remoção de nós vermelhos não implicam em violação das regras !
- Em outras palavras, como nenhum nó vermelho é considerado na altura preta de um nó, a remoção de um nó vermelho não tem consequência na altura preta de qualquer outro nó.

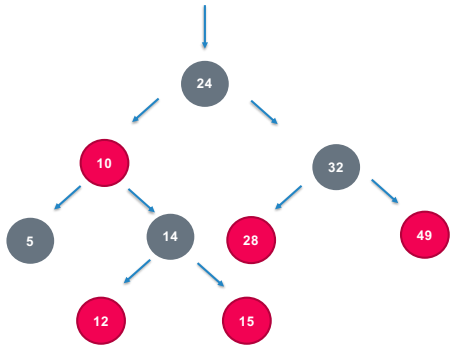
33

Exclusão em Árvores Rubro-Negras

- A remoção de um nó preto certamente causará uma violação preta do mesmo modo que a inserção de um nó preto causaria e é por isso que a inserção de nós pretos foi evitada. Além de causar violação preta, a remoção de um nó preto também pode causar uma violação vermelha.

34

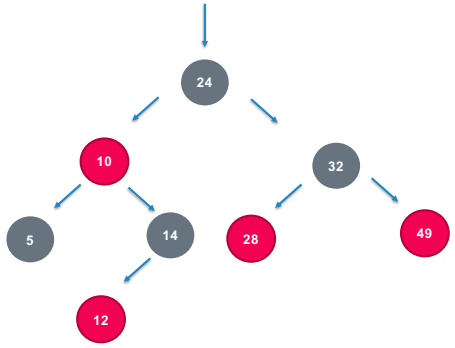
Caso 1 - Remover um nó Vermelho



- Remover o nó com valor 15.

35

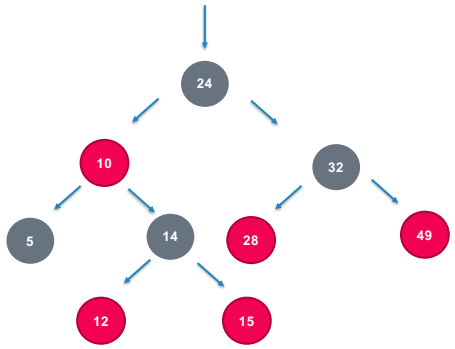
Caso 1 - Remover um nó Vermelho



- Remover o nó com valor 15.
- Realizar somente a remoção.
- Não é necessário nenhum ajuste.

36

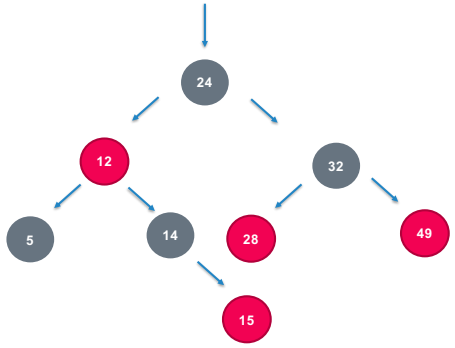
Caso 1 - Remover um nó Vermelho



- Remover o nó de valor 10.
  - Trocar id com o nó sucessor;
  - Coloração não será alterada;
  - Remover o nó;

37

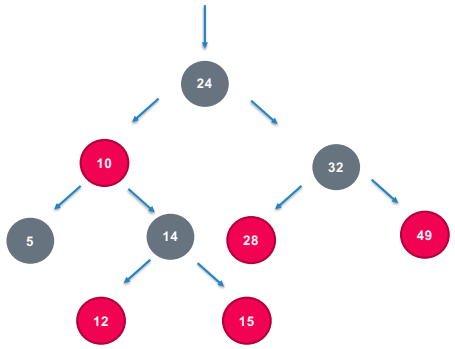
Caso 1 - Remover um nó Vermelho



- Remover o nó de valor 10.
  - Trocar id com o nó sucessor;
  - Coloração não será alterada;
  - Remover o nó;

38

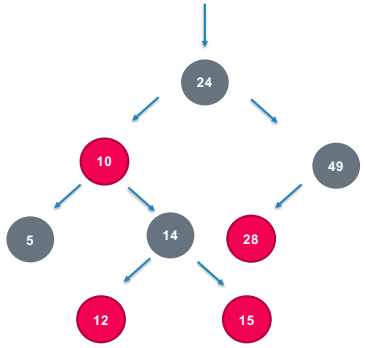
Caso 2 – Remover um nó negro com filho rubro



- Remover o nó de valor 32.

39

Caso 2 – Remover um nó negro com filho rubro



- Remover o nó de valor 32.

40

Caso 3

- O problema do duplo preto ocorre quando retiramos um nó preto;
- O duplo preto nada mais é do que uma forma de compensar a falta do nó removido na altura de preto da árvore;
- Existem 4 casos para tratar essas remoções;

41

Caso 3

Nó Removido	Substituto	Resultado
Negro	Negro	Duplo Negro
Negro	Nulo Negro	Nulo Duplo Negro
Negro	Rubro	Negro
Rubro	Rubro	Rubro
Rubro	Nulo Negro	Nulo Negro

42

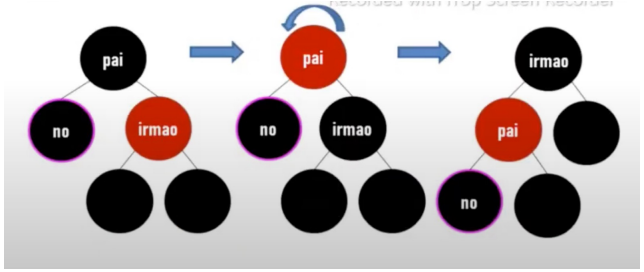
Caso 3.1 O irmão é vermelho

- Pai fica rubro;
- Irmão fica Negro;
- Rotacione o pai a esquerda
- Atualize o irmão;

- Após a rotação, o nó ainda é Duplo Negro e o novo irmão é negro, ocasionando Caso 3.2, Caso 3.3 ou Caso 3.4

43

Caso 3.1 O irmão é vermelho



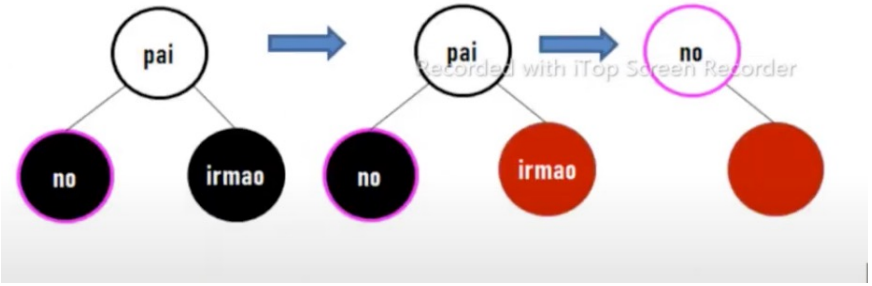
44

Caso 3.2 O irmão é preto e os filhos do irmão são pretos

- Irmão fica rubro;
- Nó passa a apontar para o pai;
- Se o nó for Rubro, basta trocar por negro ;
- Senão, o novo nó é Duplo Negro e inicia uma nova verificação

45

Caso 3.2 O irmão é preto e os filhos do irmão são pretos



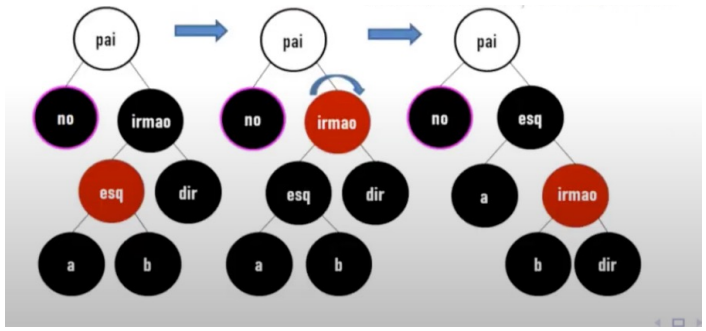
46

Caso 3.3 O irmão é preto, e o filho da direta do irmão é preto

- Filho esquerdo do irmão fica Negro;
- Irmão fica Rubro;
- Rotaciona o irmão a direita;
- Atualize o irmão;
- Após rotação: o nó ainda é DUPLO NEGRO e o novo irmão é Negro, ocasionando o Caso 3.4

47

Caso 3.3 O irmão é preto, e o filho da direta do irmão é preto



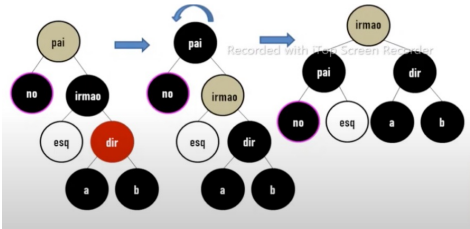
48

Caso 3.4 O irmão é preto, e o filho direito do irmão é vermelho

- Irmão copia a cor do pai;
- Pai e dir (filho do irmão) ficam negros;
- Rotacione o pai a esquerda;
- Se a raiz for vermelha, troque para preto.

49

Caso 3.4 O irmão é preto, e o filho direito do irmão é vermelho



50

Possibilidade de Casos

- CASO 3.1:
  Sempre ocasiona o CASO 3.2 ou CASO 3.3 ou CASO 3.4.
- CASO 3.2:
  Pode ocorrer sozinho, porém as verificações propagam para o ancestral.
- CASO 3.3:
  Sempre ocasiona o CASO 3.4.
- CASO 3.4:
  Pode ocorrer sozinho; e
  Após sua execução o balanceamento está concluído.

51

AVL x Árvores Rubro-Negras

- Na teoria elas possuem a mesma complexidade de inserção, remoção e busca (O(log n));
- Na prática a árvore AVL é mais rápida para buscas e mais lenta para inserção e remoção;
- As árvores AVL são mais rigidamente balanceadas do que as árvores rubro-negras, o que permite uma operação de busca mais rápida, mas também copromete o desempenho das operações de modificação;

54

AVL x Árvores Rubro-Negras

- Se a aplicação da árvore realiza de forma mais intensa operações de busca, é mais apropriado o uso de uma árvore AVL;
- Se operações de modificação são mais comuns, o melhor é usar uma árvore rubro-negra;
- Via de regra árvores rubro negras são mais utilizadas em bibliotecas.

55